Covert Channel on TCP ISN with Reverse Shell and Network-Based Detection

Marco Scarpa

Sicurezza di Internet [INQ3102230]
Università degli Studi di Padova

A.A. 2025/2026

Scenario & Scope

Scenario: a victim machine inside a corporate network has been compromised. The attacker, outside the perimeter, wants to:

- exfiltrate sensitive data without being detected
- send commands to the victim while evading the egress firewall

Covert channel choice: TCP Initial Sequence Number (ISN) field instead of the ICMP/IP.id channel shown in Module 5.

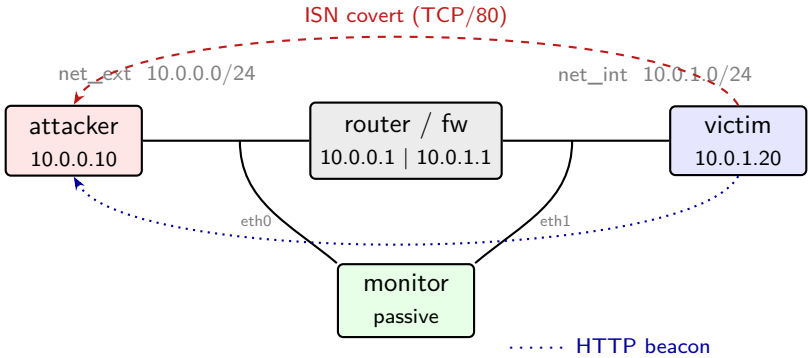
Why not ICMP?

- ICMP outbound is blocked by corporate egress filtering
- TCP/80 is almost always permitted
- More realistic in enterprise environments

Scope

- ✓ ISN covert channel (exfiltration)
- ✓ HTTP beacon (command delivery)
- ✓ Network-based detection
- × Initial delivery vector (out of scope)

Network Topology



router: TCP/80 allowed, ICMP blocked

monitor: passive – Snort 3 + Python entropy detector

Why TCP ISN? Comparison with the Course Example

	IP.id (M5 example)	TCP ISN (this lab)
Protocol field	IP Identification (16 bit)	TCP Sequence Number (32 bit)
Transport	ICMP	TCP
Egress realism	blocked in most enterprises	TCP/80 almost always allowed
Payload capacity	2 bytes/packet	2 bytes/packet (low 16 bits)
Detection signal	IP.id entropy	ISN entropy + SYN-only rate
Covert throughput	$\sim$2 B/s	$\sim$2 B/s
Complexity	Low	Medium

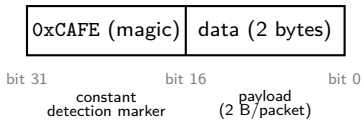
Key insight: normal ISNs are pseudo-random (RFC 6528) – structured data in the ISN field creates a measurable statistical anomaly detectable via Shannon entropy analysis.

Technology Mapping

Technology	Module	Role
Kathará multi-zone topology	M3	Network segmentation: external / internal / monitor
Scapy – TCP ISN crafting	M4, M5	Covert channel encoding and HTTP beacon C2
iptables stateful filtering	M7	Corporate egress: TCP/80 allowed, ICMP blocked
Snort 3 custom rules	M8	Signature-based detection of SYN anomalies
Python entropy detector	M4, M8	Anomaly-based detection via Shannon entropy

ISN Encoding – How It Works

ISN field structure (32 bits):



Victim (sender):

- Splits output into 2-byte chunks
- Encodes each chunk into the low 16 bits of the ISN
- Forges a TCP SYN to attacker:80 via Scapy
- Sends one SYN per chunk – no handshake ever completes

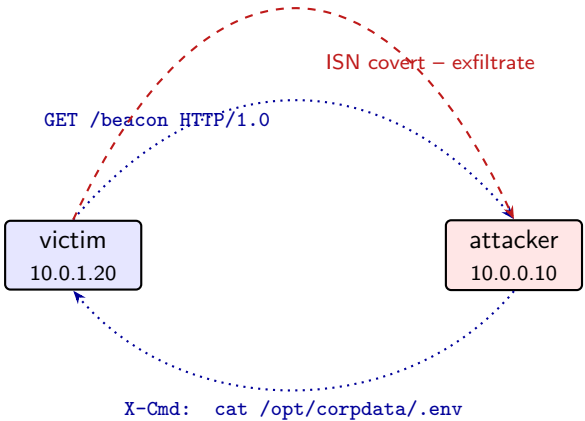
Attacker (receiver):

- Sniffs incoming SYNs on port 80
- Checks for the `0xCAFE` marker in the high 16 bits
- Extracts the low 16 bits and reassembles the byte stream
- Ignores all other TCP traffic

Anti-RST trick:

- The kernel would normally send a RST to the forged SYN
- An iptables rule on both sides drops outgoing RST packets
- This keeps the forged SYN alive long enough to be sniffed

C2 Command Delivery via HTTP Beacon



Victim loop:

1. `GET /beacon` every 10 s
2. Read `X-Cmd` from response
3. Execute command locally
4. Exfiltrate output via ISN

Why HTTP?

- TCP/80 permitted by firewall
- Looks like legitimate browsing
- Connection initiated from inside

iptables Egress Filtering

Firewall policy (router):

- Default policy: **DROP** all forwarded traffic
- **Allow** TCP/80 outbound from internal to external network
- **Block** ICMP outbound from internal network
- **Allow** return traffic for established sessions (stateful)
- **Deny** any connection initiated from outside

What this enforces:

× Victim **cannot** ping the attacker
⇒ ICMP covert channel infeasible

✓ Victim **can** reach attacker:80
⇒ TCP ISN channel works

✓ External hosts **cannot** initiate connections inbound

Verified in Demo 1:
`ping` fails, `curl http://10.0.0.10/` succeeds.

Detection: Snort 3 Signature-Based Rules

Rule 1 – ISN covert channel:

- Matches TCP packets with **SYN flag only** (no ACK) directed to port 80
- A legitimate HTTP client sends 1 SYN then completes the three-way handshake
- **3+ SYN-only from the same source in 10 s** without handshake completion is anomalous
- Fires even in idle – the covert polling generates SYNs continuously

Rule 2 – C2 HTTP beacon:

- Matches HTTP GET requests to port 80
- **2+ GETs from the same source in 30 s** to the same external host is anomalous
- Normal browsing hits many different hosts; a beacon always contacts the same one
- Rate is steady and predictable

Configuration note: Scapy-crafted packets carry invalid checksums. Snort must disable checksum validation (`checksum_eval = 'none'`) to process them – lab-specific requirement, not a real-world issue.

Key insight: Snort detects the *presence* of the channel, not individual data transfers. Both rules fire even with no active exfiltration.

Detection: Shannon Entropy on ISN Values

Principle:

- Shannon entropy H: randomness of a byte sequence
  (0 = constant, 8 = perfectly random)
- **Normal ISNs** (RFC 6528): pseudo-random
  $\Rightarrow$ $H \approx 7.5$–8.0 bits/byte
- **Covert ISNs**: `0xCAFE` fixed in high 16 bits
  $\Rightarrow$ high-byte $H \approx 0$–2 (near-constant)
  $\Rightarrow$ full $H \approx 3.5$–5.5 (structured data)

Results on our captures:

	Covert	Normal
Full H	3.9–5.6	6.4–6.5
High H	1.0–4.7	5.5–5.7
Status	**SUSPICIOUS**	normal

Strongest signal: High $H \approx 0$
The magic marker `0xCAFE` is constant across all covert packets – zero entropy contribution from the upper bytes.

Limitation: an attacker could randomize the marker per session to raise entropy.

Method: read pcap $\rightarrow$ filter SYN-only $\rightarrow$ compute H in windows of 30 ISNs $\rightarrow$ flag sources below $H < 6.0$.

Results: Snort 3 Alerts

Live output during attack (idle + exfiltration of `/opt/corpdata/.env`):

```
05/12-13:00:09 [**] [1:1000001:1]
  "[COVERT] High-rate SYN-only to TCP/80 - possible ISN covert channel" [**]
  [Priority: 0] {TCP} 10.0.1.20:54878 -> 10.0.0.10:80

05/12-13:00:09 [**] [1:1000001:1]
  "[COVERT] High-rate SYN-only to TCP/80 - possible ISN covert channel" [**]
  [Priority: 0] {TCP} 10.0.1.20:54878 -> 10.0.0.10:80

05/12-13:00:30 [**] [1:1000002:1]
  "[C2] Periodic HTTP beacon to external host" [**]
  [Priority: 0] {TCP} 10.0.1.20:53118 -> 10.0.0.10:80
```

sid:1000001 – fires on every SYN burst (exfiltration chunks)
Rate increases during active exfiltration vs idle polling

sid:1000002 – fires every $\sim$30 s
Steady rate regardless of payload – it is the beacon itself

Results: Shannon Entropy Detector

Command: `python3 detector.py /hosthome/covert_session.pcap`

```
ISN Entropy Analysis -- Covert Channel Detection
Threshold: 6.0 bits/byte  (normal ISN ~7.5+, covert ISN ~3.0-5.0)

Source IP      Window  #ISN  Full H  High H  Low H   Status
------------------------------------------------------------
10.0.1.20      0-29    30    5.6481  4.7736  4.8736  *** SUSPICIOUS ***
10.0.1.20      15-44   30    5.1262  3.1832  5.2106  *** SUSPICIOUS ***
10.0.1.20      30-59   30    3.9586  1.0000  4.9172  *** SUSPICIOUS ***
10.0.1.20      45-74   30    4.7540  2.4055  5.2151  *** SUSPICIOUS ***

ALERT: 4 window(s) flagged -- Suspect source: 10.0.1.20
Key indicator: High H near 0 -> fixed 0xCAFE in upper 16 bits of ISN
```

Full H well below 6.0 threshold in all windows
$\Rightarrow$ structured (non-random) data in ISN field

High H = 1.0 in window 30–59 is the strongest signal
$\Rightarrow$ `0xCAFE` dominates the upper byte distribution

Comparison: IP.id vs TCP ISN Covert Channel

Aspect	IP.id (M5 example)	TCP ISN (this lab)
Realism	Low – ICMP often blocked	High – TCP/80 almost always allowed
Payload	16 bits (IP.id field)	16 bits (low half of 32-bit ISN)
Throughput	$\sim$2 B/s	$\sim$2 B/s
Detection signal	IP.id anomaly (non-random sequence)	ISN entropy drop + SYN-only rate
Snort rule	`alert icmp` + payload match	`flags:S` + detection_filter
Evasion	Randomize IP.id per packet	Randomize magic marker per session
Complexity	Low	Medium (anti-RST rules needed)

Limitations & Security Considerations

Limitations:

- **Throughput:** ∼10 B/s – impractical for large files
- **ISN randomization:** RFC 6528-compliant kernels make the magic marker statistically anomalous by design
- **NIC offloading:** TSO may alter Scapy-crafted packets – disable with `ethtool -K eth0 tx off`
- **Checksum:** Scapy packets need `checksum_eval = 'none'` in Snort – not an issue in real deployments

Possible mitigations:

- Deep Packet Inspection on TCP SYN ISN distributions
- Statistical baselining of ISN entropy per source
- Anomaly threshold on SYN-only rate (no handshake completion)
- Egress TLS inspection to detect unusual traffic patterns

References:

- RFC 6528 – ISN Generation
- Course Modules M3, M4, M5, M7, M8
- Snort 3 Documentation – snort.org